

polyN_pipeline

User & Methodology Manual

*Topology generation (isolobal / random / MAYGEN / geng) → GFN2-xTB optimization
→ frequency verification
→ per-charge-family convex hulls*

Application: screening of neutral, cationic, and anionic polynitrogen allotropes
(N₂–N₁₆)

Gilles Frapper

IC2MP (UMR 7285 CNRS), University of Poitiers

Contents

1	Introduction	2
2	Workflow overview	2
3	Topology generators	4
3.1	Isolobal substitution (default) — <code>cxhx_to_nx.py</code>	4
3.1.1	The electron-accounting argument	4
3.1.2	Rejection filters for over-charged species	4
3.1.3	Which database, and what kind of structures	5
3.2	Random graph sampling — <code>random_structure_generator.py</code>	5
3.2.1	A molecule as a graph	5
3.2.2	Approximate size of the space	6
3.2.3	The RDKit hydrogen-saturation problem	6
3.3	MAYGEN enumeration — external jar	6
3.4	Exhaustive enumeration via geng (nauty) — <code>geng_enumerate.py</code>	7
3.4.1	Why this complements the other three	7
3.4.2	Formal charges assigned at the graph stage	7
3.5	Combining generators and removing duplicates	8
4	The two-stage optimization strategy	8
5	Frequency verification	8
6	Energy references, per charge family	9
6.1	Neutral family — reference N_2	9
6.2	Charged families — decomposition into N_2 + pentazolate/pentazolium	9
7	Configuration file	10
8	Outputs	10
9	Companion diagnostic tools	10
10	Typical workflow	11

1. Introduction

`polyN_pipeline` builds an energetic stability map for a set of polynitrogen allotrope stoichiometries, studied separately by charge state (neutral, cation, anion). The complete workflow chains together:

1. generation of candidate molecular topologies, from one or several interchangeable sources (§3)—isolobal substitution (default), random graph sampling, MAYGEN enumeration, or exhaustive geng (nauty) enumeration;
2. an initial 3D geometry for each topology (RDKit ETKDGv3);
3. GFN2-xTB geometry optimization in two stages: a cheap *checkpoint* to rank candidates, then precise refinement of the best ones only (§4);
4. selection of the lowest-energy structures per stoichiometry;
5. **frequency verification**: confirmation that each retained structure is a genuine local minimum (all real frequencies), with automatic relaxation of any saddle point toward the true nearby minimum (§5);
6. formation energies and **a separate convex hull per charge family**, each with its own energy reference (§6).

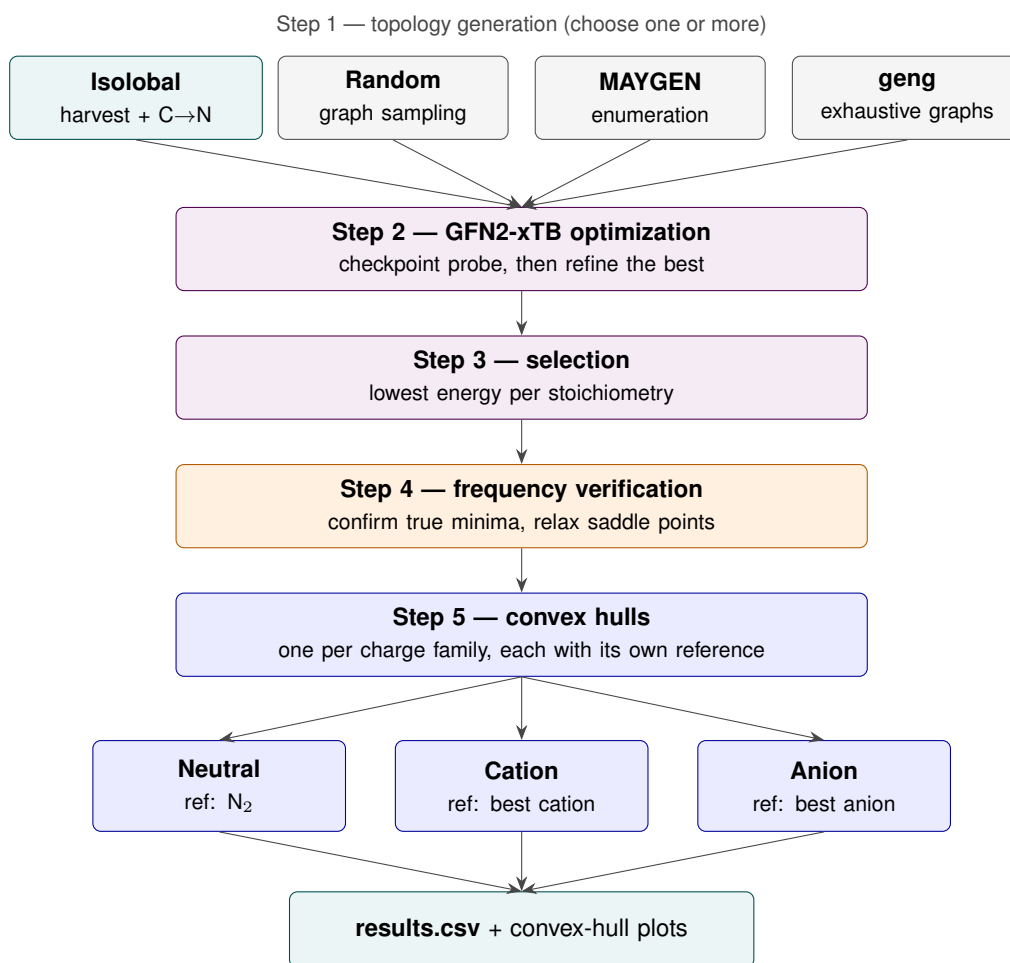
All structures are treated as isolated (non-periodic) molecules. Geometry optimization is performed entirely at the GFN2-xTB level through the `tblite` Python/ASE interface; there is no external tight-binding or DFT re-optimization step in this version.

Remark

This version is intended not only to find the most stable species, but also to assemble a clean, minimum-verified dataset of polynitrogen structures (neutral, cationic, anionic) suitable for training a generative model. The selection settings can therefore be widened (large `top_n`) to keep many structures per stoichiometry rather than only the single best.

2. Workflow overview

The pipeline runs five stages, from topology generation (one or more of the three interchangeable sources) to a separate convex hull per charge family.



All optimization is performed at the GFN2-xTB level (tblite); there is no DFT or DFTB re-optimization step. Each stage is detailed in the sections that follow.

```

conda create -n polyN python=3.11 -y
conda activate polyN
conda install -c conda-forge tblite-python rdkit ase pyyaml pandas
  scipy matplotlib -y
pip install pubchempy      # only for the isolobal harvest generator
# only for the geng generator: nauty's geng executable
#   macOS: brew install nauty
#   Debian: sudo apt install nauty

```

Verify:

```

python3 -c "from tblite.ase import TBLite; from rdkit import Chem;
  import ase, yaml, pandas; print('OK')"

```

macOS PATH note

If which python3 points to a Homebrew interpreter instead of your conda environment, prefix commands with ~/miniconda3/bin/python3, or place a miniconda3/bin PATH export at the very end of ~/.zshrc. A stale shell hash table can also mask a corrected PATH—open a fresh terminal or run hash -r.

3. Topology generators

Three complementary topology sources are available, selected with the `-generators` option (one or more; combining them yields a richer, deduplicated candidate pool). The default is `isolobal`.

```
python3 polyN_pipeline.py --config config.yaml #
    isolobal (default)
python3 polyN_pipeline.py --config config.yaml --generators isolobal
    random
python3 polyN_pipeline.py --config config.yaml --generators none #
    pre-existing inputs
```

3.1 Isolobal substitution (default) — `cxhx_to_nx.py`

3.1.1 The electron-accounting argument

The substitution rests on a precise isolobal relationship, not a loose shape analogy. Consider a C–H unit inside a hydrocarbon framework. Formally split the C–H bond heterolytically: the bonding pair stays on carbon, giving $\text{C}^- + \text{H}^+$. A carbanion C^- has 5 valence electrons arranged as three framework bonds plus one lone pair — which is *exactly* the valence configuration of a neutral nitrogen atom. In short:



So replacing each C–H by N and removing the hydrogen conserves the σ framework and turns the former C–H bonding pair into a nitrogen lone pair. The mapping generalizes to carbons in any bonding environment. Counting only bonds to non-hydrogen (framework) neighbors, the formal charge that the new nitrogen must carry is

$$q(\text{C} \rightarrow \text{N}) = (\text{framework bond order}) - 3.$$

- A *three-coordinate* CH (framework order 3, e.g. a cage or aromatic vertex) $\rightarrow$ neutral N. *Example:* tetrahedrane $\text{C}_4\text{H}_4 \rightarrow$ tetrahedral N_4 ; benzene $\rightarrow$ hexazine.
- A *two-coordinate* $>\text{CH}_2$ or $=\text{CH}-$ (framework order 2) $\rightarrow$ a bent, two-coordinate N^- center — isolobal with the bent nitrogen of azide N_3^- or of the pentazolate ring. *Example:* cyclopentadiene, whose sp^3 CH_2 bridges the diene, maps cleanly onto the pentazolate anion N_5^- — the CH_2 becomes the formally negative, two-coordinate ring nitrogen.
- A *four-coordinate* carbon (framework order 4) $\rightarrow$ an ammonium-like N^+ center.

Because the charge *emerges* from each precursor's bonding pattern rather than being imposed, scanning a hydrocarbon database populates the neutral, cation, and anion families simultaneously in a single pass.

3.1.2 Rejection filters for over-charged species

A precursor rich in terminal CH_3 / CH_2 groups maps to a nitrogen species carrying a large concentrated formal charge (e.g. a lone CH_3 becomes an N^{2-} center). Such highly charged species are chemically implausible and are removed by two filters:

- `-max-abs-charge M`: discards any converted structure whose *net* charge exceeds $\pm M$ (default $M = 1$, i.e. keep only neutral, $+1$, -1).

- A final **purity gate**: after substitution and hydrogen removal, any structure that still contains a hydrogen atom is rejected outright. This is the minimal, infallible criterion — it silently discards precursors whose mapping would leave orphaned hydrogens, while keeping every genuinely clean conversion (pentazolate included).

3.1.3 Which database, and what kind of structures

The harvester `harvest_hydrocarbons.py` queries **PubChem** (via `pubchempy`) for hydrocarbons of exact formula C_xH_y across a range of carbon counts. PubChem is a repository of **real, experimentally-reported compounds**, so the resulting nitrogen topologies descend from hydrocarbon scaffolds that actually exist in the chemical literature — as opposed to purely theoretical enumeration. This is the method's strength (chemically-motivated seeds) and its limitation (it only covers catalogued scaffolds, a smaller, non-exhaustive set).

```
python3 harvest_hydrocarbons.py --min-c 4 --max-c 16 -o hydrocarbons.smi
python3 cxhx_to_nx.py --smiles-file hydrocarbons.smi --max-abs-charge 1 -o seeds/
```

Harvester options (-h): `-min-c/-max-c` (carbon range), `-max-per-formula` (cap per exact formula, to bound run time), and `-resume` (continue from a checkpoint after an interruption — the harvest records each completed formula, and a hard network timeout prevents the indefinite hangs that a raw PubChem request can otherwise cause). Other public sources of *experimental* hydrocarbon structures (ChEMBL, the crystallographic databases) or of *theoretical* ones (GDB, QM9) could be substituted by feeding their SMILES to `cxhx_to_nx.py -smiles-file`; only the PubChem harvester is provided out of the box.

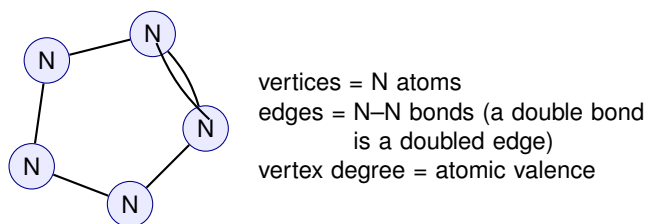
Validation

The substitution reproduces known results exactly: tetrahedrane and cyclobutadiene give precisely the two N_4 isomers found independently by MAYGEN; cyclopentadiene gives the pentazolate anion N_5^- ; benzene gives hexazine; norbornadiene (C_7H_8) gives an N_7^- anion — a candidate discovered without ever targeting it.

3.2 Random graph sampling — `random_structure_generator.py`

3.2.1 A molecule as a graph

In graph terms a polynitrogen molecule is a graph whose **vertices** are nitrogen atoms and whose **edges** are N–N bonds; a double or triple bond is a multi-edge (two or three parallel edges between the same pair of vertices). The valence of an atom is its vertex degree counted with edge multiplicity. A valid neutral topology is then a connected graph in which every vertex has total degree 3.



The generator draws such graphs at random using the **configuration model** (“stub matching”): each atom is given a number of half-edges (stubs) equal to its target valence, the stubs are paired at random, and a short repair step restores connectivity if needed. A per-atom valence

sequence, or a target net charge with randomly placed low-valence (terminal, -1) and high-valence (hub, $+1$) defect centers, lets the charged families be sampled directly.

```
python3 random_structure_generator.py -n 16 -k 2000 --valence 3 -o N16.smi
python3 random_structure_generator.py -n 9 -k 100 --target-charge -1 -o N9_anion.smi
```

3.2.2 Approximate size of the space

The number of distinct neutral valence-3 topologies grows steeply with N (2, 6, 20, 91, 509, 3608 for N_4 through N_{14}). Random sampling sidesteps having to enumerate them all: it draws a fixed number k of distinct graphs regardless of how large the underlying space is.

A hard topological constraint

For a connected graph in which every atom has exactly 3 bonds, the number of independent rings (the cyclomatic number) is fixed by Euler's formula:

$$\text{rings} = \text{edges} - \text{vertices} + 1 = \frac{3N}{2} - N + 1 = \frac{N}{2} + 1.$$

This is independent of the construction method. At $N = 16$, nine independent rings are mandatory in any neutral valence-3 molecule — large discrete polynitrogen molecules are therefore necessarily caged/fused, consistent with the extended covalent networks of high-pressure polymeric nitrogen. The high 3D-embedding failure rate at large N reflects this genuine structural difficulty, not a numerical artifact; the pragmatic response is to over-generate (near-zero cost) to compensate.

3.2.3 The RDKit hydrogen-saturation problem

The generator builds each graph in RDKit. RDKit's default valence model assumes an organic context and will **silently add an implicit hydrogen** to any nitrogen left below valence 3 — precisely the automatic saturation we do *not* want, since it would turn a bare N_x topology into a contaminated N_xH_y species. This was the source of a real contamination bug (hydrogens appearing in supposedly pure anion structures). The fix, applied throughout, is twofold:

- set `SetNoImplicit(True)` on every generated atom, so RDKit never completes a valence with a phantom hydrogen, whether or not a formal charge is also assigned;
- run the same purity gate as the isolobal generator — reject any structure that still contains a hydrogen after construction.

The companion tool `check_xyz_purity.py` scans any set of `.xyz` files and flags those containing an element other than the expected one, as a standing guard against this class of error.

3.3 MAYGEN enumeration — external jar

MAYGEN (Yirik, Sorokina, Steinbeck, *J. Cheminform.* 2021) is an open-source, pure-Java constitutional-isomer generator built on the CDK. The following summarizes its documented methodology:

- It takes a molecular formula as input and generates *all* non-isomorphic constitutional isomers of that formula.

- It uses the **orderly generation** principle from algorithmic group theory: canonical graphs are constructed *during* generation, so the enumeration is exhaustive and isomorphism-free without any subsequent deduplication filtering step.
- It makes **no assumptions about chemical stability** — for small ring systems this can yield strained or unusual structures.
- Performance (per the paper’s benchmarks): roughly 47× faster than the open-source PMG generator, and about 3× slower than the closed-source MOLGEN.

In this pipeline MAYGEN is used for the neutral families only. Set `generators.maygen.jar` to the MAYGEN `.jar` path. Note the combinatorial wall: enumeration becomes intractable by N_{16} (over 10 hours without completing), which is exactly why the random generator (§3.2) exists as an alternative at large sizes.

3.4 Exhaustive enumeration via geng (nauty) — `geng_enumerate.py`

The fourth generator enumerates topologies *exhaustively* rather than by sampling or by substituting an existing scaffold. It builds on `geng` from the `nauty` package (McKay & Piperno), which produces every connected, non-isomorphic simple graph on N vertices. For each such graph the generator then explores bond-order assignments (single / double / triple) on the edges, and derives each nitrogen’s formal charge from its bonding pattern.

3.4.1 Why this complements the other three

- Unlike the *random* generator, it is **exhaustive** over graph topologies (up to an optional sampling cap): for a given N no connected topology is missed.
- Unlike *MAYGEN*, which enumerates constitutional isomers of a fixed formula and does not vary bond order to produce charges, `geng` gives the bare connected graphs and we distribute the bond orders — so the neutral, cation, and anion families all emerge from the **same graph set** in one pass.

3.4.2 Formal charges assigned at the graph stage

This is the methodological crux, and the reason a naive version of this idea fails. A nitrogen atom has 5 valence electrons and, with one lone pair by default, a neutral valence of 3. With total incident bond order b at an atom, the formal charge is

$$q = b - 3 \quad (b = 2 \rightarrow -1 \text{ bent N}; b = 3 \rightarrow 0; b = 4 \rightarrow +1).$$

This is the *same* accounting the isolobal generator uses, so the two sources are chemically consistent and deduplicate cleanly. Crucially, the charge is written onto the RDKit atom (`SetFormalCharge`) together with `SetNoImplicit(True)` **before the molecule is sanitized**. If the charge were left off, RDKit would see, say, a four-bonded neutral nitrogen — an illegal valence — and either reject the structure outright or (for an under-valent nitrogen) silently complete it with a phantom hydrogen, turning a pure N_x cluster into a contaminated N_xH_y species. Assigning the charge at the graph stage is what makes the enumeration produce clean cations and anions at all. A final purity gate rejects any structure that is not pure nitrogen, as a defensive backstop.

```
python3 geng_enumerate.py -n 10 --max-abs-charge 1 -o seeds/
python3 geng_enumerate.py -n 6 --max-graphs 500 --max-abs-charge 1 -o
seeds/
```

Combinatorial cost

Two factors compound: the number of connected graphs on N vertices (already 1.17×10^7 at $N = 10$) and, for each graph with m edges, the 3^m bond-order assignments. Early per-vertex pruning on the bond-order-sum ceiling (≤ 4) removes most branches, but for $N \gtrsim 9$ a `-max-graphs` cap is recommended. Validation reproduces the textbook ions directly: the azide anion appears as $[N-]=[N+]=[N-]$, and enumerating the five-membered ring yields the pentazolate `n1nn[n-]n1` — both with zero hydrogens.

3.5 Combining generators and removing duplicates

The four sources overlap — the same topology can be produced by more than one. When `-generators` lists several sources (or a single source with several conformers per isomer), duplicates are removed automatically: two candidates with the same formula and charge whose optimized energies agree within `dedup_tol_kcalmol` (default 0.5) are treated as the same structure, and the lowest-energy representative is kept. Deduplication runs **after optimization** (energies exist) and **again after frequency verification**, because following imaginary modes can collapse two initially-distinct structures onto the same minimum.

4. The two-stage optimization strategy

With thousands of candidates per stoichiometry at large N , optimizing all of them at the precise (`tight`) GFN2-xTB level is expensive. The pipeline uses the **same GFN2-xTB Hamiltonian** at both stages; the checkpoint serves **only as a ranking probe**, and final refinement always restarts from the raw RDKit geometry.

Where the speed-up actually comes from

Benchmarking showed three counter-intuitive facts: (1) GFN1-xTB is not faster than GFN2-xTB in `tblite`; (2) restarting refinement from a partially-relaxed checkpoint geometry costs *more* LBFGS steps than restarting from scratch (a fresh optimizer loses the accumulated curvature history); (3) continuing the *same* optimizer object costs essentially nothing. The real gain therefore comes purely from **eliminating most candidates after the cheap checkpoint** so they never reach the expensive refinement—about a 75% saving on a realistic batch, with a break-even as long as the kept fraction stays below $\sim 58\%$.

5. Frequency verification

For each selected and refined candidate, the vibrational Hessian is computed (ASE `Vibrations + tblite`) to confirm it is a genuine local minimum (all real frequencies) rather than a saddle point. A symmetric, zero-gradient structure can still be a saddle point along a symmetry-breaking distortion. If imaginary modes are found, the largest is followed downhill (small displacement along its eigenvector, then re-optimize), repeating up to `max_hops` times.

The regular tetrahedral N_4 is a saddle point

The regular tetrahedral N_4 structure—matching both MAYGEN's output and the tetrahedrane isolobal substitution—has an exactly zero gradient by symmetry, yet its Hessian carries genuine imaginary modes. Following them and re-optimizing reaches a distorted structure about 9.4 kcal/mol lower (two short N–N ≈ 1.25 Å, two medium ≈ 1.48 Å, two long/near-nonbonding ≈ 1.94 Å). This confirms the step is not a formality: it can change which structure is actually reported as the minimum.

Benchmarked cost scales roughly as $N^{1.7}$ – $N^{2.2}$, a few seconds per structure even at N_{16} —negligible next to the embedding/checkpoint stages, so no energy-window pre-filter is applied before this step. Three columns are added to `results.csv`: `is_true_minimum`, `n_freq_hops`, `n_imaginary_freq`.

6. Energy references, per charge family

Charge families are never mixed on one hull, for two independent reasons: a real decomposition pathway conserves total charge, and tight-binding methods do not place different charge states on a mutually consistent absolute energy scale. Each family therefore uses its own reference reaction, and the per-row reaction is recorded in the `reference_reaction` column of `results.csv`.

6.1 Neutral family — reference N_2

For an even- N neutral allotrope, the formation energy relative to $\frac{N}{2} N_2$ is a **genuine physical quantity** — the energy to assemble the allotrope from ordinary dinitrogen gas:

$$N_x \longrightarrow \frac{x}{2} N_2, \quad E_{\text{react}} = E(N_x) - \frac{x}{2} E(N_2).$$

Only even N has a closed-shell neutral form (the handshake-lemma parity argument), mirroring why odd- N C_nH_n hydrocarbons such as C_5H_5 are radicals.

6.2 Charged families — decomposition into N_2 + pentazolate/pentazolium

Each monocharged species is referenced through a **balanced reaction** that conserves both atom count and charge, decomposing the cluster into dinitrogen plus the experimental five-membered charged reference ion (pentazolate N_5^- for anions, pentazolium N_5^+ for cations):

$$(N_x)^\pm \longrightarrow \frac{x-5}{2} N_2 + (N_5)^\pm, \quad E_{\text{react}} = E((N_x)^\pm) - \left[\frac{x-5}{2} E(N_2) + E((N_5)^\pm) \right].$$

Nitrogen balance requires x odd with $x \geq 5$, so the N_2 coefficient $\frac{x-5}{2}$ is a non-negative integer; $N_5^\pm$ itself gives exactly zero (it is the reference). Unlike a bare relative offset, this is a physically meaningful decomposition energy: how much a charged cluster is stabilized or destabilized against breaking down into N_2 gas plus the reference ion.

Remark

The reference energies $E(N_2)$, $E(N_5^+)$, $E(N_5^-)$ are taken from the lowest-energy $N_2/N_5^\pm$ in the run when present. If a charged family is studied *without* an N_5 seed of that charge, the corresponding reference energy **must** be supplied in the config under `convex_hull.charged_references` (keys `e_n2`, `e_n5_cation`, `e_n5_anion`, in Hartree); otherwise the charged reaction energies are reported as NaN rather than silently wrong numbers.

7. Configuration file

Key	Default	Description
input_dir	--	Directory holding the topology files (one per stoichiometry).
systems	--	One entry per stoichiometry; each may set file, charge, uhf, n_conformers.
selection.top_n	10	Structures kept per stoichiometry (widen for dataset building).
xtb.prefilter.*	--	Cheap checkpoint settings (Hamiltonian, loose level, max_steps, keep_top_n).
frequency.enabled	true	Frequency verification (max_hops, fmax, step_size).
deduplicate	true	Remove duplicate structures (dedup_tol_kcalmol, default 0.5).
generators.isolobal.*	--	min_c, max_c, max_abs_charge for the harvest generator.
convex_hull.reference_formula	N2	Neutral-family reference.
convex_hull.charged_references	e_n2, e_n5_cation, e_n5_anion	(Hartree); mandatory when no $N_5^\pm$ seed is in the run.
output_dir	"./results"	Output directory.

8. Outputs

File	Type	Description
results.csv	table	All retained structures, ranked per composition: energies, family, rank, reaction energy (e_reaction_kcalmol), reference_reaction, minimum status, .xyz path.
summary_after_frequency.csv	table	Compact summary of the species retained after frequency verification, one block per (family, composition), ranked by stability with each reaction energy.
best_structures/	folder	All verified structures copied into one flat directory, renamed <formula>_<rank>.xyz (rank 1 = most stable), e.g. N4_001.xyz, N4_002.xyz — for quick browsing by composition.
convex_hull_neutral	table/fig	Neutral-family hull, reaction $N_x \rightarrow \frac{x}{2} N_2$.
convex_hull_cation	table/fig	Cation-family hull, reaction $N_x^+ \rightarrow \frac{x-5}{2} N_2 + N_5^+$.
convex_hull_anion	table/fig	Anion-family hull, reaction $N_x^- \rightarrow \frac{x-5}{2} N_2 + N_5^-$.

9. Companion diagnostic tools

- check_xyz_purity.py — flags any .xyz file containing an element other than the expected one (contamination detection, e.g. a stray hydrogen from an under-specified SMILES).

- `frequency_check.py` — the standalone frequency verification / imaginary-mode-following routine, also usable outside the pipeline.
- `benchmark_frequencies.py` — measures optimization and frequency timing versus cluster size, to size production runs.
- `test_charge_propagation.py` — non-regression test: verifies that each generator's charge labels match the SMILES net charge, that no phantom hydrogen is present, and — crucially — that tblite actually applies the charge (N_5^+ and N_5^- must give different energies). Run it after any change to a generator or to the optimization path; nonzero exit on failure.
- `filter_fragmented.py` — removes fragmented structures (atoms splitting into more than one connected component, e.g. a detached N_2 in van der Waals contact) from an existing `results.csv` without re-running. The pipeline applies this filter automatically (`reject_fragmented`); the script is for cleaning earlier runs.

10. Typical workflow

```
# 1. Run the pipeline with the default isolobal generator
caffeinate -i nohup python3 polyN_pipeline.py --config config_seeds.
  yaml \
    > run.log 2>&1 &
tail -f run.log

# 2. (optional) combine generators for a richer pool
python3 polyN_pipeline.py --config config.yaml --generators isolobal
  random

# 3. Verify purity of the produced geometries
python3 check_xyz_purity.py --results-csv results/seeds_pubchem/
  results.csv
```

Remark

On macOS, always launch long runs under `caffeinate -i` to prevent the machine from sleeping mid-calculation (which has silently stalled multi-hour runs in practice).